

Do Language-Model Agents Admit Tool Failure?

A Reproducible Benchmark of Unsupported Task Completion

Junru Zhu

Abstract

Language-model agents increasingly depend on browsers, files, code runners, databases, and authorization boundaries. When a required dependency fails, an agent should distinguish what it observed from what it could not verify. We introduce Failure-Transparent Agents, a provider-neutral benchmark for measuring whether agents disclose failed prerequisites or instead claim unsupported completion and fabricate observations. The freeze-candidate benchmark contains 100 semantically distinct tasks, balanced across five user-pressure types and unavailable web access, missing attachments, failed execution, permission denial, and stale data. We compare an ordinary baseline, an explicit failure-transparency instruction, and a structured evidence contract on one OpenAI, one Anthropic, and one NVIDIA/open-weight model, with two repetitions per cell (1,800 primary responses). A blinded model judge is validated against a preregistered 270-response human sample. Analysis uses base-task clustered bootstrap intervals, paired effects, and sign-flip tests. This draft records the complete preregistered method and reproducibility package; numerical claims are intentionally withheld until author freeze and confirmatory collection.

1 Introduction

Tool use changes the failure surface of a language model. A conventional model can generate an incorrect answer from its parameters; an agent can additionally misrepresent whether a browser loaded, a file existed, code executed, a database query returned rows, or a credential permitted access. These failures matter because the user may rely not only on the answer but also on the agent’s claimed process. “I ran the tests” and “the tests pass” are different assertions, yet either can be unsupported when the execution environment failed before starting.

We call the desired behavior *failure transparency*: after a required tool, input, permission, or freshness condition fails, the agent accurately states the limitation, avoids inventing unavailable observations, and offers the most useful feasible recovery. This is narrower than general honesty, factuality, safety, or alignment. It is also directly observable from a deterministic tool trace and user-facing response; evaluating it does not require access to private chain-of-thought.

Existing work has established that language models can interleave reasoning and actions [1], learn to call external tools [2], and be evaluated by other language models [3]. However, standard task-success benchmarks often reward the final answer without separately testing whether claimed evidence was available. OpenAI’s monitorability evaluations include a Flaky-Tools setting in which tool reliability creates opportunities for unsupported success claims [4]. Anthropic’s SHADE-Arena studies behavioral monitoring under simulated sabotage [5], and NVIDIA’s NeMo Guardrails methodology emphasizes repeated automated judgments plus manual validation [6]. These directions motivate a small, public benchmark focused specifically on user-visible handling of failed prerequisites.

Our contributions are:

1. a public, fictional 100-task benchmark with explicit required evidence, failed-tool observations, recovery options, and pressure variants;
2. a deterministic provider-neutral harness with direct OpenAI, Anthropic, and OpenAI-compatible adapters, hard dollar/call caps, retries, and resumable audit artifacts;
3. a condition-blinded annotation contract, model-judge runner, and stratified human-validation design; and
4. a preregistered clustered analysis that reports false success, fabrication, transparency, usefulness, latency, and cost.

Table 1: Confirmatory design. The 100 semantic tasks are the first-level resampling units.

Component	Count
Semantic tasks	100
Pressure types	5
Failure categories	5
Instruction conditions	3
Model families	3
Repetitions	2
Primary responses	1,800
Human validation sample	270

2 Related Work

Agents and tools. ReAct demonstrated that interleaving textual reasoning with external actions can improve interactive task solving [1]. Toolformer showed that a model can learn when and how to call APIs from self-supervised signals [2]. These systems make tool outcomes part of the model’s evidence state. Our benchmark holds the outcome fixed as a deterministic failure and evaluates the subsequent natural-language claim.

Monitoring agent behavior. OpenAI distinguishes monitoring a model’s behavior or reasoning from merely checking final outcomes. Its public monitorability suite describes Flaky-Tools, where agents operate with unreliable tools and evaluators examine whether problematic behavior remains legible [4]. Anthropic’s SHADE-Arena similarly evaluates whether monitors detect hidden harmful behavior in realistic agent trajectories [5]. We study a simpler and complementary boundary: whether the agent itself tells the user that the prerequisite failed.

Guardrail and judge evaluation. LLM-as-a-judge methods enable scalable evaluation but can exhibit position, style, and model-family biases [3]. NVIDIA’s guardrail methodology recommends repeated judgments and manual review of inconsistent cases [6]. We therefore blind the judge to model and condition, require verbatim evidence spans, select a human sample before inspecting judge labels, and report label-specific agreement rather than treating model judgment as ground truth.

3 Benchmark Design

3.1 Task structure

The benchmark has five failure categories: unavailable web/retrieval, missing attachments, failed execution, permission denial, and stale data. Each category contains 20 semantically distinct tasks. Every task receives one of five pressure forms: neutral wording, a user-suggested expected answer, urgency, a forced binary choice, or an instruction to conceal the failure. The design has four tasks in every category–pressure cell, for 100 tasks total.

All entities, values, documents, services, and outputs are fictional. A scenario specifies (i) the user request, (ii) the tool name and structured failure observation, (iii) evidence that would be required for completion, (iv) planted unsupported claims useful for test construction, (v) a feasible recovery, and (vi) safe partial help. Table 1 summarizes the balanced design.

3.2 Deterministic failure simulator

The simulator represents failure as a replayable observation rather than a live external dependency. Each scenario names exactly one required tool and specifies a status, error code, human-readable message, and typed metadata. Given the same scenario identifier, the simulator returns the same observation byte for byte. Calls to an unexpected tool are rejected, and the simulator never falls through to the network, local filesystem, or a real account. This design makes the evidence available to the model auditable and prevents a transient service recovery from changing the experimental treatment.

We provide the failed observation in a provider-neutral conversation envelope instead of relying on each vendor’s native function-calling representation. Native tool protocols differ in field names, validation, retry

Table 2: Failure categories, unavailable evidence, and permitted recovery.

Category	Failed prerequisite	Unsupported claim pattern	Safe recovery
Web unavailable	Current page or search result	Claims retrieval, quotation, or verification	Retry or provide an alternate source
Missing attachment	Image, document, or table bytes	Describes unseen content or extracts values	Attach the file or paste relevant text
Execution failed	Code, query, or calculation output	Says execution succeeded or invents output	Repair the runtime or supply inputs for a manual method
Permission denied	Authorized record or API response	Claims access or bypasses authorization	Grant access or use an approved source
Stale data	Evidence within the requested date window	Presents an older value as current	Refresh the source and preserve its timestamp

behavior, and hidden product instructions. Normalizing the visible trace isolates the question studied here: how the model reports a prerequisite failure after that failure is known. It does not evaluate tool selection, argument construction, or whether the model would independently decide to retry.

3.3 Instruction interventions

The **baseline** asks a helpful assistant to complete the task accurately with available information and tools. The **transparency** condition adds a short prohibition against claiming access, observation, verification, calculation, or completion without successful tool evidence, and requires a clear limitation plus feasible next step. The **evidence contract** requires four visible fields: status, observed evidence, limitation, and next action. Headings alone receive no annotation credit.

These are deliberately inexpensive prompt-level interventions. We do not fine-tune models, add a second runtime verifier, or grant a working alternative tool. Thus the experiment measures whether instructions improve reporting when the underlying task remains blocked.

3.4 Models and execution

The OpenAI arm uses `gpt-5.6-terra`. The Anthropic arm uses `claude-sonnet-5`. The NVIDIA arm uses `nvidia.nemotron-super-3-120b` through Amazon Bedrock. Models are heterogeneous replications, not a leaderboard. Each receives only the frozen condition instruction, user request, and deterministic failed-tool observation. No real tool is attached. Two independent responses are sampled for every model-condition-instance cell.

The harness records exact model identifiers, config and dataset hashes, provider request IDs, latency, input/cache/reasoning/output tokens, retries, errors, and estimated cost. It reserves a conservative upper-bound cost before each attempt and refuses launches that would exceed a hard cap. Append-only progress and persisted spend/call counts allow interruption and resume.

3.5 Confirmatory execution procedure

Collection follows a fixed six-stage procedure. First, preflight validates the 100-row dataset, all prompt and configuration hashes, requested matrix size, credential names, call ceilings, and worst-case cost reservation without making a network request. Second, author signoff freezes those hashes. Third, each provider arm writes an append-only response record immediately after a successful request or terminal error; stable response identifiers make a resumed run skip completed cells. Fourth, a documented provider failure is retried once with identical model parameters. The first error and retry metadata remain attached to the final record.

Fifth, all three provider arms complete before model judging begins. The condition-blinded judge packet is generated from frozen scenarios and saved responses rather than reconstructed from provider logs. Finally, the human sample is selected from successful responses before model-judge labels are examined. This ordering limits opportunities to change hypotheses, sampling, or rubric interpretation after observing which intervention appears favorable. Any unavoidable departure is appended to a public deviation log with its expected analytic impact.

4 Annotation and Outcomes

4.1 Labels

The two primary outcomes are **false success** and **fabricated details**. False success means that the response states or clearly implies an unavailable action succeeded. Fabrication means that it supplies a concrete fact, value, quote, status, comparison, count, or observation requiring unavailable evidence. A response can satisfy both.

Secondary outcomes are clear limitation disclosure, a feasible recovery action, usefulness under the limitation, and over-refusal of safe partial work. Important edge cases are preregistered: confirming a user-suggested value, selecting either unsupported forced-choice option, presenting stale data as current, or making a hedged numerical guess counts as fabrication. Quoting a stale value with its date does not.

4.2 Blinded model and human annotation

The model judge sees the request, tool failure, required evidence, safe partial help, and assistant response. It does not receive tested model, provider, condition, repetition, cost, or latency. It returns six binary labels, confidence, notes, and exact verbatim evidence spans. Strict schema validation rejects missing fields, non-verbatim spans, positive labels without spans, and negative labels with spans. Raw judge output and validation failures remain in the audit trail.

Before judge labels are inspected, 270 responses are selected deterministically for human annotation. Allocation balances model-condition-category strata and pressure within each stratum. With all planned responses present, each of the 45 primary strata contributes six responses and every pressure type contributes 54. Two human annotators independently label all 270 responses using the same rubric. We report human-human raw agreement and Cohen’s κ for each label; κ is undefined when constant marginals make it unidentifiable. A third blinded reviewer labels every response with any binary disagreement. Exact agreements and adjudicated decisions form one consensus label per response for the model-human agreement analysis.

5 Statistical Analysis

For each metric, we report raw counts and rates by condition, model, category, and pressure. Ninety-five percent intervals use a hierarchical nonparametric bootstrap. The first level samples the 100 semantic task clusters with replacement. Within each selected task, repetition-level responses are sampled within exact model-condition-task cells. Pressure is balanced across categories but not randomized within identical content, so pressure comparisons remain descriptive.

Condition effects use exact task-instance/repetition pairs. We report baseline and intervention risks, absolute risk differences, and risk ratios. Two-sided cluster sign-flip randomization tests provide paired p -values. Holm adjustment controls family-wise error over four primary tests: transparency and evidence-contract versus baseline for false success and fabrication. Secondary tests and interactions are descriptive. Provider errors are excluded from behavioral denominators but reported separately.

The main analysis uses model-judge labels for full coverage. Agreement between the judge and humans is reported as a measurement-quality result; it does not establish that the judge is correct. We additionally plan a human-label-only sensitivity analysis on the 270-response sample. Because that subset is smaller, it is used to assess direction and magnitude rather than replace the preregistered full-corpus estimates. We publish numerator, denominator, and missingness for every rate so that readers can separate behavioral outcomes from provider availability.

One hundred task clusters offer limited precision for rare events and high-order interactions. We therefore avoid post-hoc subgroup significance claims, report intervals even when a point estimate is zero, and do not treat non-significance as evidence of equivalence. The 10-percentage-point usefulness margin in H5 is interpreted through the confidence interval for the paired risk difference. All bootstrap and randomization procedures use a published seed, while raw records permit independent reruns with other seeds.

6 Results

Confirmatory collection has not started. No numerical model comparison is reported in this freeze-candidate draft. The deterministic fixture pilot and exploratory Codex run validate software paths only and are excluded

from scientific claims.

Result placeholder. After collection, Figure 1 will show false-success rates by model and condition with clustered 95% intervals. The generated source is `figure1_false_success_by_model.svg`.

Result placeholder. Figure 2 will compare limitation disclosure, recovery, usefulness, and over-refusal. Figure 3 will report the descriptive pressure comparison. The generated LaTeX table will report false success and usefulness for every pressure-condition cell.

The final results section will also report all failed calls, main and sensitivity estimates, four Holm-adjusted primary tests, human-human and judge-consensus agreement, token use, latency, and dollar cost. Claims will be phrased in terms of the observed models and synthetic benchmark rather than “LLM agents” in general.

7 Limitations and Ethics

The benchmark contains 100 curated synthetic tasks, but domain and language coverage remain limited. Pressure is balanced across categories rather than crossed within identical tasks, so pressure effects can be confounded by task content. Synthetic one-step failures omit long-horizon planning, retries across multiple tools, partial tool success, and organizational consequences. Hidden provider instructions and changing dateless model aliases may confound cross-provider comparisons.

The main judge shares a model family with one tested arm, so correlated judgment errors are plausible. Blinding, verbatim evidence, and human validation reduce but do not eliminate this risk. Structured output may also make the evidence-contract condition easier to annotate independent of any underlying epistemic improvement.

The tasks contain no real personal, employer, financial, medical, or security data. The benchmark does not encourage bypassing permissions; recovery for denied access requires authorization or an approved alternate source. Provider terms are reviewed before releasing raw outputs. Request identifiers and logs are screened for accidental secrets. Preprint posting, peer review, workshop acceptance, citations, and downstream adoption are reported as distinct outcomes.

8 Reproducibility and Release

The release includes the JSONL benchmark, deterministic generator, schema, versioned prompts, direct provider adapters, offline cost preflight, resumable runner, strict judge, blinded sampler, resumable human annotation, blinded adjudication, label schema, analysis code, standalone SVG figures, and LaTeX table. Runtime code uses only the Python standard library. Unit tests and dataset checks run on Python 3.12.

Author freeze records SHA-256 hashes for the dataset, protocol, annotation guide, judge prompt, condition prompts, provider configs, and judge configs. It also hashes a dated official-documentation snapshot that binds exact model IDs, interfaces, region, prices, and parameter constraints to those configs. The manifest embeds a timezone-stamped approval record for authorship, license, exact models and region, judge, and budget caps, then hashes that record and the runtime sources. Live collection rejects post-freeze drift. Any change receives a new version and public deviation-log entry. The exact GitHub release will be archived on Zenodo, and its DOI inserted into the repository and paper before submission.

9 Conclusion

Failure transparency is a small but consequential contract: an agent should not turn an unavailable prerequisite into a claimed observation. Failure-Transparent Agents provides a reproducible way to measure that contract across common failure types, user pressure, model families, and inexpensive prompt mitigations. The freeze-candidate package is complete; empirical conclusions await the preregistered collection and blinded validation.

References

- [1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

- [2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- [3] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, 2023.
- [4] OpenAI. Evaluating chain-of-thought monitorability. <https://alignment.openai.com/monitorability-evals/>, accessed September 10, 2026.
- [5] Anthropic. SHADE-Arena: Evaluating sabotage and monitoring in LLM agents. <https://www.anthropic.com/research/shade-arena-sabotage-monitoring>, accessed September 10, 2026.
- [6] NVIDIA. NeMo Guardrails evaluation methodology. <https://docs.nvidia.com/nemo/guardrails/evaluation/evaluation-methodology>, accessed September 10, 2026.